

UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI FISICA

CORSO DI LAUREA TRIENNALE IN FISICA

ANALISI E SIMULAZIONI DEI READ OUT TREE DEL CIRCUITO INTEGRATO TIMESPOT1

FELICE FRANCARIO

Matricola: 919369

Relatore: Prof. Valentino Liberali

Correlatori: Dott. Luca Frontini,

Dott. Alberto Stabile

Anno academico 2021-2022

Indice

Introduzione **iii**

1 Progetto Timespot **1**
1.1 High-Luminosity LHC 1
1.2 Architettura di Timespot1 2
1.2.1 Analog Front End 4
1.2.2 Time to Digital Converter(TDC) 5
1.2.3 Read Out Tree (ROT) 6

2 Errori di circuito nel ROT e possibili cause **9**
2.1 Trasmissione dei dati 9
2.1.1 Funzione del clock nel trasferimento dei dati 11
Setup time e hold time 11
2.2 Osservazioni sperimentali 12
2.3 Effetti contribuenti alle osservazioni sperimentali 13
2.3.1 Jitter 14
2.3.2 Skew 15
2.3.3 IR Drop 17

3 Simulazioni e misure sul Read-out tree **19**
3.1 Simulazioni sul ROT 21
3.1.1 Simulazioni post-sintesi 21
3.1.2 Simulazioni post-layout 24
3.2 Simulazioni sul ROT evidenziando il duty cycle del clock 30
3.3 Analisi e misure relative all'effetto IR Drop 31

4 Conclusioni **35**

Bibliografia **37**

Introduzione

Timespot1 è un chip prototipo di un frontend per pixel 4D dotato di 1024 canali indipendenti, progettato in tecnologia commerciale CMOS a 28 nm, per applicazioni nella fisica delle alte energie. Il circuito integrato Timespot1 oggetto di studio nasce come parte di una collaborazione fra 10 unità INFN, il progetto TIMESPOT (TIME and SPace real time Operating Tracker), con obbiettivo di sviluppare un rivelatore 4D ad alta precisione temporale e spaziale per HL-LHC (High-Luminosity Large Hadron Collider).

Lo scopo di questa tesi è di trovare la causa ed una possibile soluzione all'attuale problema di corruzione dei dati riscontrato durante il test su alcune uscite ad alta frequenza del circuito integrato Timespot1. A questo scopo si effettuano simulazioni utilizzando netlist backannotate e testbench scritti con linguaggi di descrizione hardware. Si procede analizzando diversi aspetti del circuito tra cui il jitter, lo skew tra i diversi clock e la caduta di tensione (I-R drop) nei vari nodi del ciruito di read-out.

La tesi è strutturata in 3 capitoli di cui il primo descrive il funzionamento delle principali componenti e l'architettura del chip Timespot1; nel secondo capitolo, vengono descritti i risultati osservati nei test sperimentali e alcuni possibili effetti contribuenti ad essi; infine nell'ultimo capitolo si trovano i risultati delle varie simulazioni digitali effettuate nel tentativo di analizzare le possibili cause delle osservazioni sperimentali.

Capitolo 1

Progetto Timespot

1.1 High-Luminosity LHC

Il Large Hadron Collider (LHC) è l'acceleratore di particelle più grande e potente al mondo. È stato avviato per la prima volta il 10 settembre 2008 e rimane l'ultima aggiunta al complesso di acceleratori del CERN a Ginevra[1]. L'LHC è costituito da un anello di 27 chilometri di magneti superconduttori con una serie di strutture di accelerazione per aumentare l'energia delle particelle lungo il percorso.

Negli anni 2025-27, è pianificato un upgrade dell'acceleratore di particelle LHC con l'obiettivo di aumentare la luminosità integrata di un fattore 10 rispetto all'attuale versione dell'LHC [2][3]. In questo periodo verrà fatto un upgrade dell'acceleratore di particelle e dei rivelatori in termini di performance e resistenza alle radiazioni. Il High-Luminosity Large Hadron Collider (HL-LHC), che dovrebbe essere operativo dall'inizio del 2029, consentirà ai fisici di studiare in modo più approfondito i meccanismi noti, come il bosone di Higgs, e di osservare possibilmente nuovi fenomeni rari.

Un tipico rivelatore di nuova generazione dovrà soddisfare le seguenti condizioni minime[4]:

- Dimensioni del pixel inferiore a 55 µm
- Risoluzione temporale di almeno 50 ps
- Resistenza alla radiazione di 24 MGy

Dalle sopra citate necessità è nato il progetto TIMESPOT (TIME and SPace real time Operating Tracker) frutto di una collaborazione fra dieci unità di ricerca INFN.[5]

Il sistema è composto da un sensore, di 1024 pixel tridimensionali a stato solido, costruito in silicio e associato ad un ASIC (Application Specific Integrated Circuit) mediante il processo di bump-bonding (Figura 1.1)[6]. Il sensore è costituito da una matrice di 32×32 pixel, ciascuno dei quali di misura 55×55 μm^2. All'interno del chip ogni canale è costituito da un front-end analogico ed un TDC (Time to Digital Converter).

L'ASIC collegato al sensore viene denominato Timespot1.

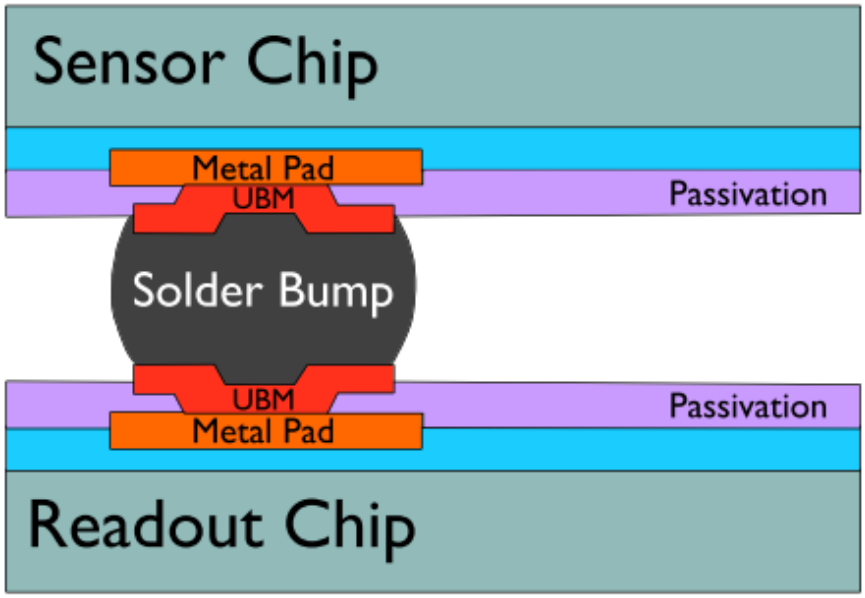


Figura 1.1: Intereconnessione tra il sensore e il Readout chip di Timespot1

1.2 Architettura di Timespot1

Il chip Timespot1 è stato progettato in tecnologia 28nm che permette di ottenere un ottimo rapporto costo-prestazioni. La tecnologia 28 nm favorisce, con la sua miniaturizzazione, un'architettura in cui ciascun circuito di read-out di singolo pixel è dotato di un sotto-circuito, TDC (Time to Digital Converter), in grado di eseguire misure temporali.

Il chip è costituito da 1024 canali di read-out, organizzati in 4 quadranti, in una matrice di 32×32 pixel. I canali in ingresso sono suddivisi in 4 gruppi da 256,

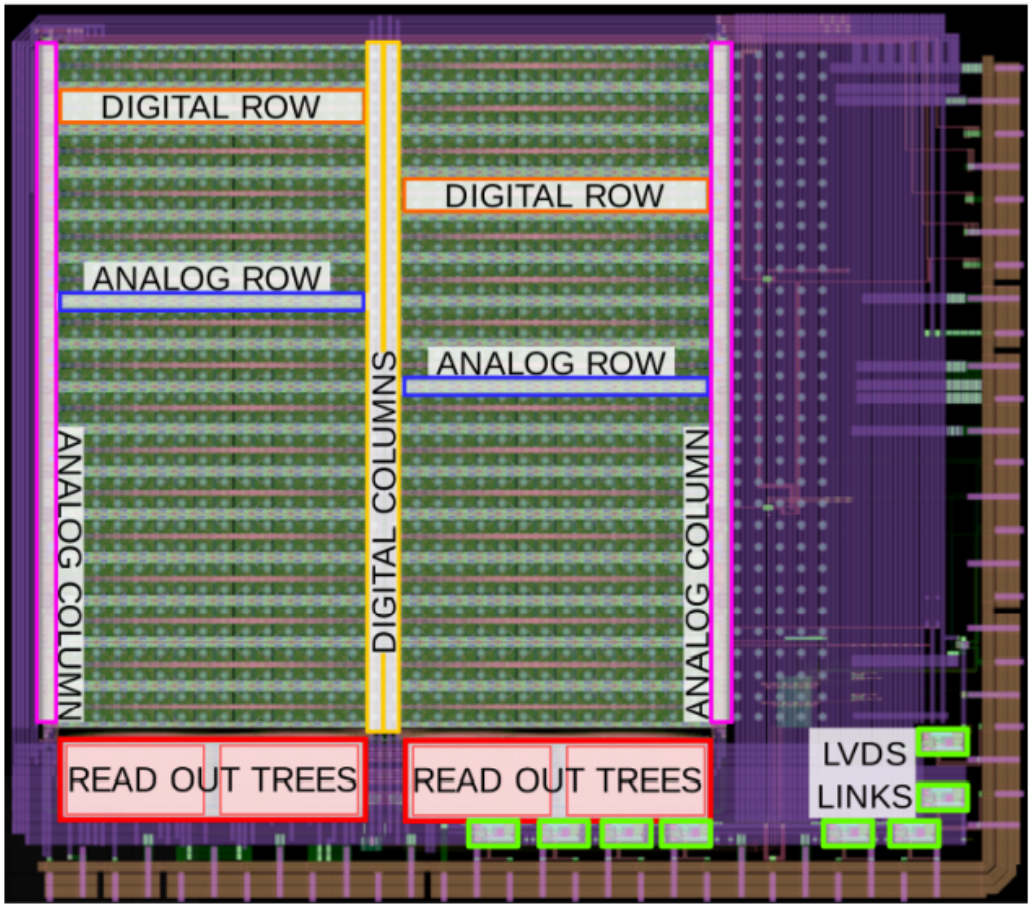


Figura 1.2: Layout del Chip Timespot1 con in evidenza le sue componenti più importanti

ognuno collegato a un blocco di Read Out Tree (ROT), necessario per raccogliere i dati dai canali attivi (si veda la Figura 1.2). Ogni canale di input è dotato di un sistema di front-end e di un TDC (Time to Digital Converter) che misura il tempo di arrivo e il tempo sopra soglia delle particelle incidenti sul sensore, rispettivamente Time of Arrival (TA) e Time Over Threshold (TOT). Ogni TDC presenta due uscite: una in cui viene trasmesso un segnale di data valid e l'altra in cui viene trasmessa l'informazione della misura di TA e TOT serializzata ad una frequenza di 160 MHz.

Una volta raccolti i dati dai canali attivi, il ROT assegna un timestamp globale, poi li trasmette a uno delle 2 FIFO connesse ai driver LVDS (Low Voltage Differential Signal). Sommando la velocità di trasmissione degli 8 LVDS, ciascuno con dati in arrivo a una velocità di 1.28 Gbit/s, si ha una banda complessiva di 10.24 Gbit/s (Si veda la Figura 1.3).

L'Array di 32×32 pixel è suddiviso in 2 blocchi specchiati di 16×32. Il dominio digitale viene separato da quello analogo rendendoli indipendenti. La trasmissione di dati dal TDC avviene nelle colonne digitali presenti nella parte interna della struttura fra i 2 blocchi. La parte Analoga si svolge nelle colonne analoghe presenti sui 2 bordi esterni del chip.

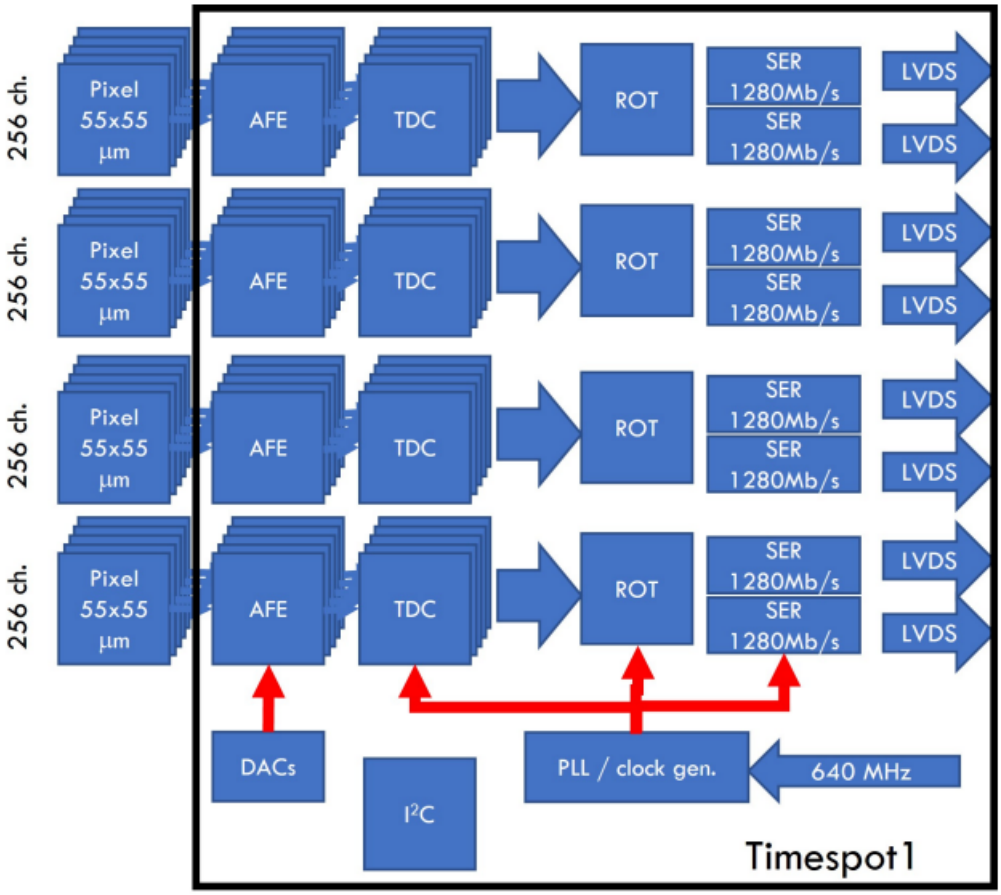


Figura 1.3: Schema a blocchi del chip Timespot1

1.2.1 Analog Front End

Ogni canale è costituito da un CSA (Charge Sensitive Amplifier) come primo stadio e un discriminatore come secondo stadio. L'ingesso di ogni CSA è connesso al bond-pad del sensore e a un circuito inietta cariche per generare impulsi di correnti di test. Lo stadio del CSA produce un output di tensione proporzionale

alla carica integrata dei segnali che riceve in input. L'architettura del CSA rende proporzionale l'input di carica al segnale di output del TDC: Time Over Threshold (TOT).

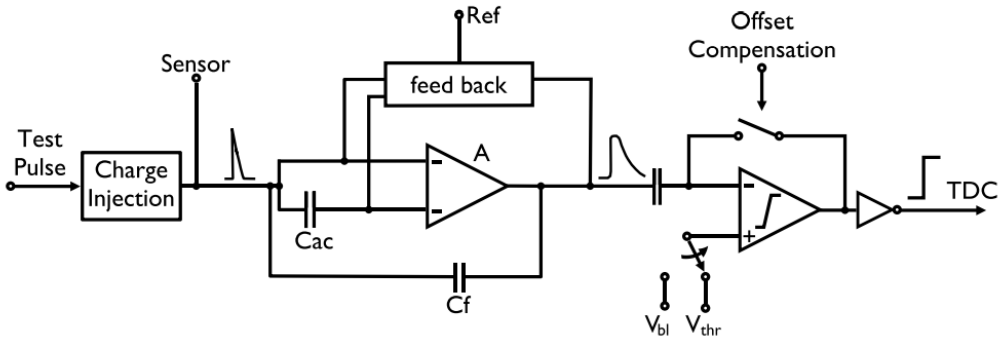


Figura 1.4: Schema della parte analogica del Front-End

Nel secondo stadio, il discriminatore produce gli impulsi digitali che fungeranno da input per il TDC. In questo stadio, i segnali amplificati dal CSA vengono confrontati con una data soglia di tensione, in modo da generare un segnale digitale con fronte di salita ripido.

1.2.2 Time to Digital Converter(TDC)

Il TDC misura il Time of Arrival (TA) del segnale che viene generato all'interno del AFE in termini di fase rispetto a un clock di riferimento di 40 MHz. Allo stesso tempo il TDC misura anche il Time Over Threshold (ToT) del segnale per correggere il Time Walk e di conseguenza migliorando la precisione della misura del TA. Il TDC è basato su un'architettura Vernier (Figura 1.5), con due Digital Controlled Oscillator (DCO) che lavorano a due frequenze diverse e controllano un contatore veloce. Mediante un Coincidence detector Circuit (CC), la conversione viene fermata quando si rilevano due fronti di salita contemporanei.

Il tempo misurato (TA) è dato da:

$$TA = (n_1 - 1)T_1 - (n_2 - 1)T_2$$

dove T_1 e T_2 sono i periodi dei due DCO mentre n_1 e n_2 sono i rispettivi contatori. La risoluzione teorica dell'architettura di Vernier è data da:

$$r = T_1 - T_2$$

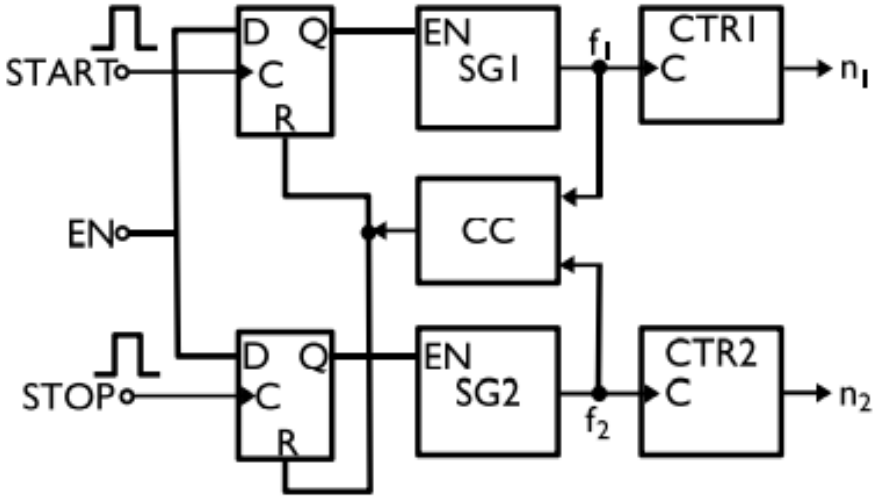


Figura 1.5: Architettura del TDC

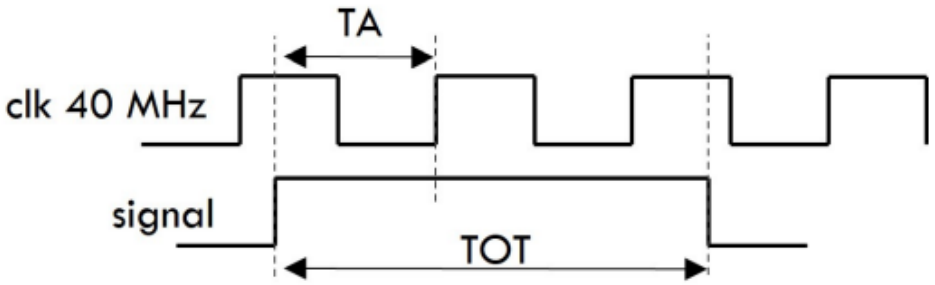


Figura 1.6: Il TDC misura il TA (Time of Arrival) e il TOT (Time Over Threshold) in contemporanea

1.2.3 Read Out Tree (ROT)

Per ogni *hit*, il TDC invia 23 bit (serializzati a 160 MHz) al ROT in fondo alla colonna. Questo insieme di 23 bit viene accoppiato con 9 bit bi Timestamp che indicano in che colpo di clock a 40 MHz viene registrato l'evento misurato. Il ROT poi genera le coordinate geografiche (Address di 8 bit) dei pixel di provenienza per ciascun segnale generato nel AFE. L'output del ROT è costituito da: 8 bit di Address, 23 bit dal TDC, e 9 bit di informazione Timestamp. La stringa risultante di 40 bit viene acquisita da una delle 2 FIFO (First In First Out) di 32 strati operanti ad una frequenza di 160 MHz. Le FIFO immagazzinano dati in modo che il primo

dato introdotto è il primo ad uscire, prevenendo perdite di dati in situazioni di alto flusso di particelle. Mediante il blocco *protocol*, il pacchetto di 40 bit in uscita dalla FIFO viene suddiviso in 5 byte preceduti da un byte header. Nel caso in cui non siano presenti dati in uscita dalla FIFO, il blocco *protocol* continua a trasmettere il byte di *idle*. Il byte di *idle* e *header* possono essere entrambi impostati dal protocollo I^2C. Ogni byte viene poi serializzato e convertito in Double Data Rate (DDR) per poi essere trasmesso in uscita con una velocità di 1280 Mbit/s mediante un protocollo LVDS (Low Voltage Differential Signaling).

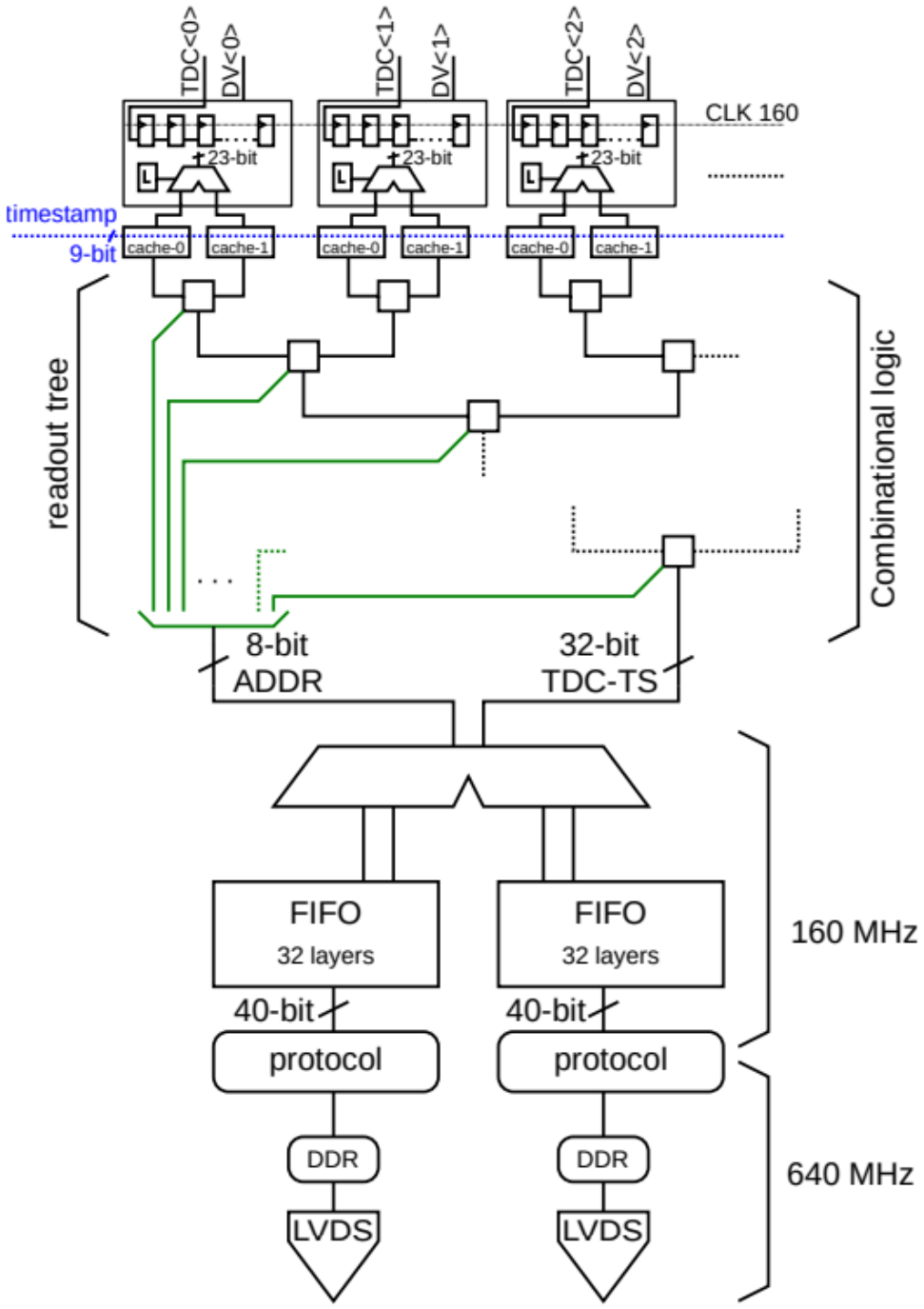


Figura 1.7: Schema del ROT

Capitolo 2

Descrizione dell'errore misurato all'uscita del circuito nel ROT e ipotesi sulle possibili cause

Lo scopo del lavoro di tesi è di avvicinarsi il più possibile alla causa del problema trovato nel funzionamento di una parte del circuito integrato: Il Read Out Tree. Infatti nella fase di test sperimentale, per la verifica del corretto funzionamento del circuito progettato, si è notato un disaccordo fra i dati in uscita rispetto ai valori attesi. Un risultato non osservato nella fase precedente di simulazione del circuito. Per descrivere bene l'errore osservato bisogna prima comprendere il funzionamento atteso del circuito.

2.1 Trasmissione dei dati

Come accennato nel capitolo precedente, i pacchetti di dati vengono trasmessi in uscita come una stringa di byte tramite il blocco protocol. Il Read-out tree è stato progettato in modo che continui a trasmettere un byte di *idle* fino a quando non riceve un dato valido da trasmettere.

I byte possono essere espressi con una sequenza di 8 bit nel sistema binario, oppure in modo più compatto con una rappresentazione esadecimale (sistema numerico in base 16). Con il sistema numerico esadecimale c'è una relazione diretta tra una cifra esadecimale e 4 cifre binarie come nella tabella 2.1.

È possibile esprimere ogni byte con 2 cifre esadecimali. Per l'*idle* si è scelto di trasmettere la parola "AB" che corrisponde alla sequenza binaria "10101011",

Tabella 2.1: Relazione tra la rappresentazione esadecimale, decimale e binaria di un numero

hex	dec	bin
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

volutamente asimmetrica come in figura 2.1.

Quando poi arriva la stringa di 40 bit da una delle FIFO del ROT allora l'idle viene seguito da un byte di *header*, corrispondente alla parola "E5" ("11100101"), che segnala l'effettivo inizio della trasmissione della serie di 5 byte di informazione del *hit*.

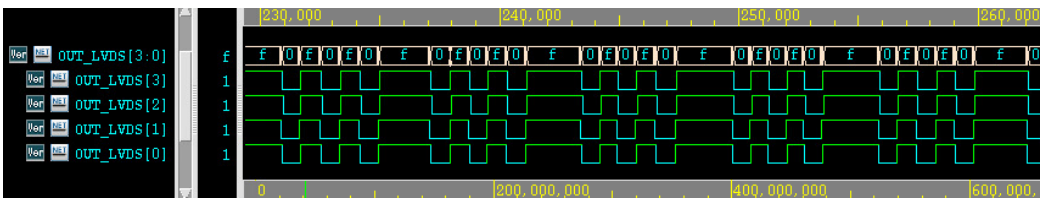


Figura 2.1: Simulazione delle uscite LVDS che trasmettono in continuo i byte dell'*idle* "AB" come atteso dal progetto

2.1.1 Funzione del clock nel trasferimento dei dati

Il clock è il segnale a forma di onda quadra con una determinata frequenza utilizzato per sincronizzare il funzionamento del circuito digitale. Nel chip di Timespot1, il trasferimento di dati avviene attraverso una logica digitale principalmente basata su un clock di frequenza 640 Hz.

Il circuito è costituito da elementi basilari di memoria noti come flip-flop. I flip-flop sono dispositivi fondamentali per circuiti che necessitano di archiviare dati in modo temporaneo. Ci sono vari tipi di flip-flop, ma per capire come il clock possa influenzare il trasferimento dei dati, ci basta focalizzare sul funzionamento dei flip-flop di tipo D (Data). Infatti quest'ultimi sono di tipo edge-triggered, ossia sensibili al fronte di salita del segnale di clock. Essi hanno un ingresso per il dato, un ingresso di sincronizzazione(clock) e un'uscita, la quale registra il valore del dato in ingresso ad ogni fronte di salita del clock. Quindi il flip-flop D si può interpretare come un sistema dotato di memoria di 1 bit. La registrazione del valore di uscita non avviene istantaneamente ma dopo un leggero ritardo di propagazione.

Setup time e hold time

In un flip-flop D, come in qualsiasi circuito digitale, esiste un certo ritardo di propagazione dal momento di arrivo del segnale di ingresso all'istante di uscita del segnale di output. In particolare, c'è un tempo che trascorre tra il fronte di salita del clock e il cambiamento del valore di uscita. Questo ritardo è dovuto dal tempo impiegato dalla logica combinatoria del dispositivo ad elaborare il segnale oltre al tempo di salita del segnale di uscita.

Il tempo necessario, dal fronte di salita del clock, affinché il dispositivo acquisisca il dato dal segnale di ingresso, viene definito come *hold time*. Se il segnale di ingresso cambia valore in questo intervallo di tempo successivo al fronte di salita del onda, il dispositivo rischia di registrare il valore successivo, con la conseguente perdita del dato precedente. Invece il *setup time* si può definire come l'intervallo di tempo precedente al fronte di salita in cui il segnale di ingresso non deve cambiare valore per la corretta interpretazione del dato [7]. Si veda la Figura 2.2.

Dall'unione dei 2 intervalli temporali di setup e hold attorno al fronte di salita del clock, si forma una zona temporale in cui il segnale di ingresso deve essere necessariamente stabile e di conseguenza si crea un limite alla frequenza massima

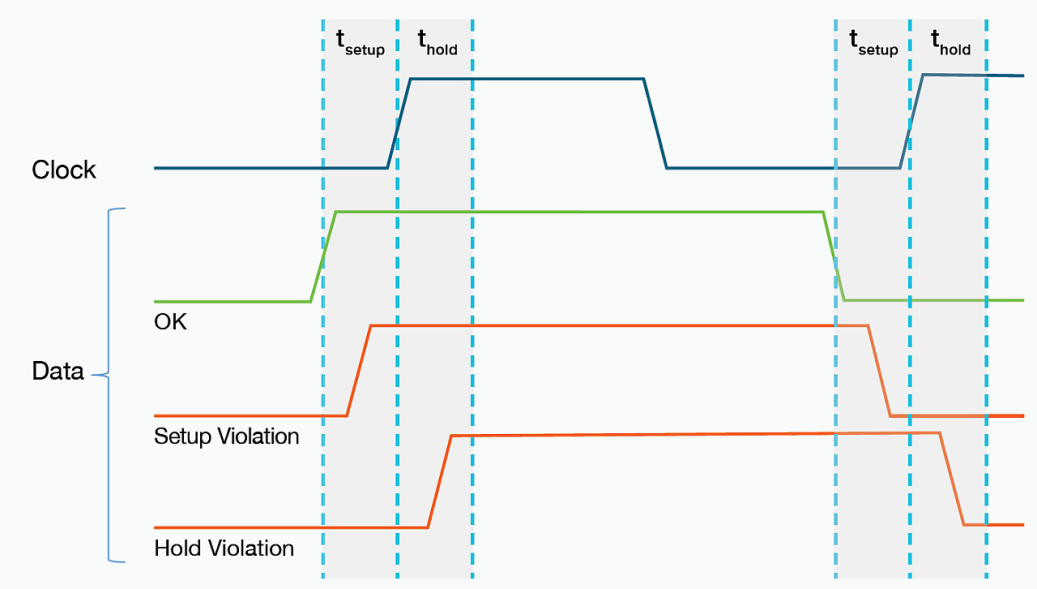


Figura 2.2: Hold time e Setup time

del clock. Infatti, per non avere un ulteriore variazione del valore di ingresso nell'intervallo di tempo , il periodo di clock deve essere sufficientemente grande da contenere il *setup time* e il *hold time.*

$$P_{clk} > t_{setup} + t_{hold}$$

2.2 Osservazioni sperimentali

Nelle simulazioni di test, descritte in dettaglio nel capitolo 3,si è verificato il corretto funzionamento del circuito senza aver trovato errori nella trasmissione dei dati. Situazione purtroppo non condivisa nei test sperimentali. Dopo la progettazione del Read out chip di Timespot1, sono stati effettuati diversi test sperimentali per verificare il corretto funzionamento del circuito fisico. Nella verifica della trasmissione dei dati, si è osservato un andamento non atteso nel segnale digitale in uscita dagli LVDS in 2 dei 4 blocchi di ROT in fondo del chip. In particolare, si è osservato una perdita di bit oltre a un restringimento degli zeri nel segnale di *idle*, come in Figura 2.3. I 2 blocchi di ROT affetti da questo problema sono quelli denominati "Bottom 0", situati nella parte interna, accanto alla colonna digitale del chip (si veda la Figura 2.4).

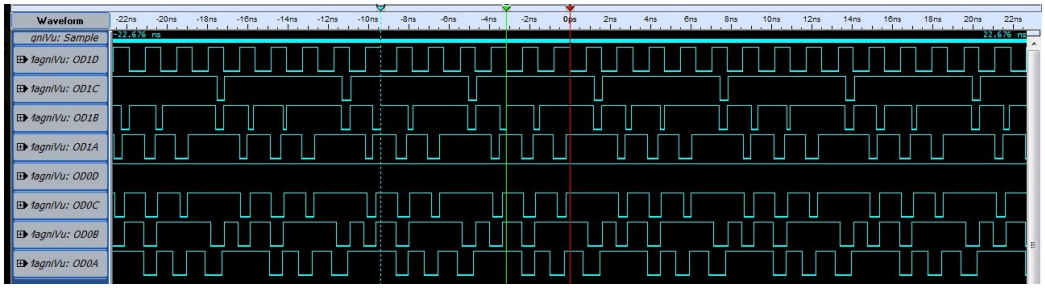


Figura 2.3: Osservazione sperimentale della trasmissione delle parole di *idle* dal chip di Timespot1

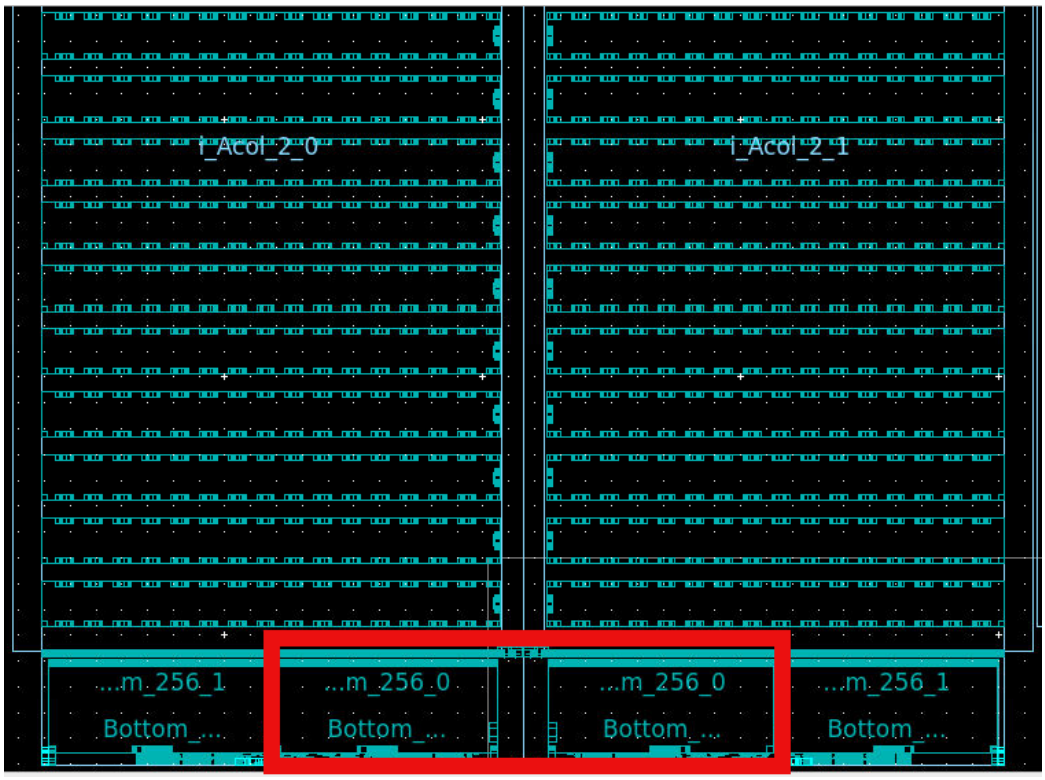


Figura 2.4: Layout del chip Timespot1, in cui sono evidenziati in rosso i "Bottom 0" problematici

2.3 Ipotesi sugli effetti fisici contribuenti alle osservazioni sperimentali

Qual è il motivo della divergenza osservata tra i risultati sperimentali e quelli simulati? Perché la discrepanza si presenta solo nei 2 blocchi ROT interni del chip? Qual è la causa del restringimento degli "0" nelle *waveform* dei segnali in uscita da questi ROT?

Per rispondere a questi quesiti abbiamo analizzato in dettaglio diversi possibili effetti contribuenti alle osservazioni sperimentali osservate.

2.3.1 Jitter

Sebbene normalmente gli uni e gli zeri di una stringa digitale di bit possano essere distinti facilmente, tali bit sono comunque rappresentati da segnali elettrici che attraversano un mondo analogico affetto da rumore. Se una quantità elevata di rumore elettrico si sovrappone ad un segnale digitale, questa può causare del jitter nella forma d'onda digitale, ovvero traslarlo nel tempo. Ciò può causare problemi se il fronte di passaggio da zero a uno (o viceversa) viene traslato così lontano da non renderlo correttamente interpretabile al punto di campionamento. Quando questo accade, il circuito di ricezione può interpretare erroneamente il valore binario codificato nella forma d'onda, introducendo così un errore di trasmissione.

Le cause principali del jitter sono:

- *Fenomeni legati al sistema*

  Si tratta di effetti che si ripercuotono sul segnale a causa del fatto che il sistema digitale si trova in un ambiente analogico.

- *Fenomeni legati ai dati*

  Si tratta di sorgenti di jitter dovute al tipo di sequenza di dati trasferiti, come ad esempio l'alterazione del duty cycle e della periodicità nelle sequenze di bit.

- *Fenomeni di rumore casuale*

  Si tratta del jitter dovuto principalmente al rumore elettronico generato dalla agitazione termica dei portatori di carica oltre al suo flusso nei semiconduttori e conduttori.

Il jitter totale di un segnale è la somma dei contributi di jitter casuali e deterministici che lo affliggono.
La componente casuale è la composizione di tutte le sorgenti di rumore casuale

che influenzano il segnale. Si considera che il jitter casuale segua una distribuzione gaussiana e quindi sia caratterizzato da una media e da una deviazione standard. La componente deterministica del jitter si ricava sommando l'anticipo di tempo massimo al ritardo di tempo massimo prodotto dalle sorgenti limitate.

Per questo lavoro di tesi abbiamo studiato, mediante simulazioni, l'effetto di un jitter casuale sul clock principale del circuito. Inoltre, si sono anche effettuate varie simulazioni studiando l'effetto di un jitter deterministico, ovvero modificando il duty cycle del clock. I risultati delle simulazioni e le conseguenti osservazioni saranno approfondite nel prossimo capitolo.

2.3.2 Skew

Oltre all'effetto del jitter casuale sul clock, è importante considerare anche i ritardi sistematici dovuti alle diverse lunghezze delle interconnessioni del circuito. Ogni linea di connessione metallica, presente tra le varie parti dell'ASIC, ha una componente resistiva e capacitiva non trascurabile. Queste connessioni possono essere approssimate,come in Figura 2.5, da una serie di coppie di piccole resistenze e capacità di $R = rdx$ e $C = cdx$, dove *r* e *c* rappresentano rispettivamente le densità lineari della resistenza e capacità del filo conduttore con unità di lunghezza dx.

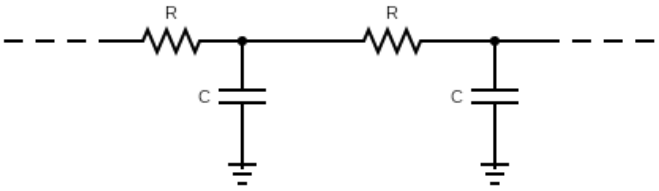


Figura 2.5: Schema di approssimazione delle componenti di resistenza e capacità delle piste metalliche del circuito

Con quest'approssimazione, si ricava l'equazione del Calore:

$$rc\frac{\partial v(x,t)}{\partial t} = \frac{\partial^2 v}{\partial x^2}$$

Risolvendo l'equazione, per una funzione gradino, si trova un output con andamento di un *error function* con argomento dipendente anche dalla lunghezza

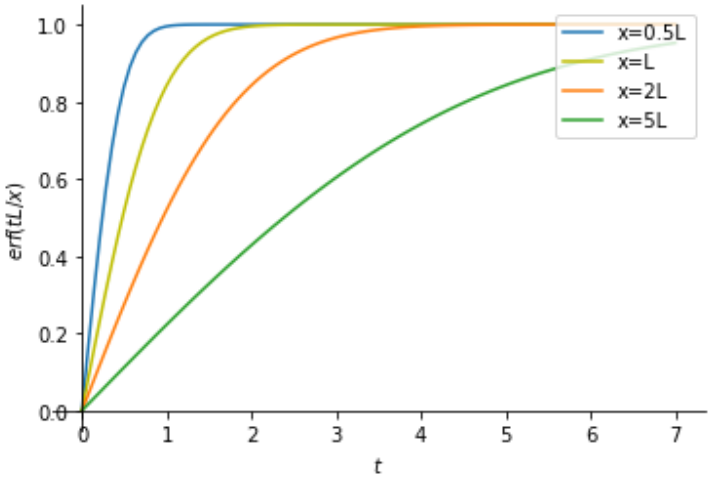


Figura 2.6: Immagine illustrativa dell'effetto di skew sul fronte d'onda di salita di un clock di onda quadra e la sua dipendenza dalla lunghezza della linea di interconnessione tra dispositivi

x della linea in unità di una lunghezza di riferimento L, come in figura 2.6 :

$$v(x,t) \approx \text{erf}\left(t\frac{L}{x}\right) = \frac{2}{\sqrt{\pi}}\int_0^{t\frac{L}{x}} e^{-z^2}dz$$

Analizzando la modifica del clock in funzione della lunghezza del filo conduttore, si può ricavare una stima del conseguente ritardo digitale:

$$t_d(x) \approx 0.7\frac{rc}{2}x^2$$

Definiamo come skew, la variazione del ritardo di ricezione di un bit tra più dispositivi, rispetto al suo istante di trasmissione. Avendo più ricevitori collegati tramite la stessa linea di interconnessione, ognuno di essi riceverà il bit inviato dalla sorgente comune dopo un differente tempo di trasmissione detto t_{tx}, questo valore varierà per ogni dispositivo a causa di fattori come la resistenza e la capacità equivalente che ognuno di essi mostra ai capi del trasmettitore. Definendo $t_{x,min}$ come tempo di trasmissione minore tra tutti i ricevitori e, $t_{x,max}$ come quello maggiore, definiamo lo skew come la misura di variazione massima del ritardo di trasmissione.

$$t_{skew} = t_{x,max} - t_{x,min}$$

Per ridurre al minimo questo fenomeno, si utilizzano delle distribuzioni *ad albero a H*, come in figura 2.7. In questo modo le distanze dal generatore di segnale

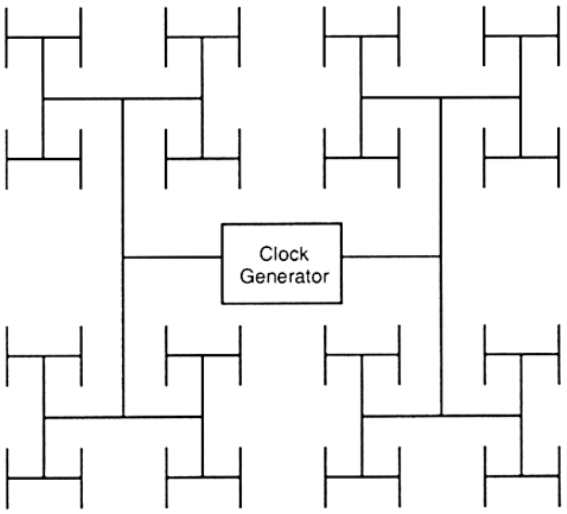


Figura 2.7: Rete di distribuzione ad albero ad H

clock presente al centro a tutti i punti di diramazione dell'albero sono uguali. In questo modo non si osserva una differenza di fase significativa tra i vari segnali di clock in arrivo a componenti di distanza simile dal centro dell'albero.

2.3.3 IR Drop

Per il corretto funzionamento del circuito integrato è necessario fornire un'alimentazione ad ogni transistor presente all'interno del chip. La tensione di alimentazione è distribuita all'interno del chip usando delle interconnessioni di metallo. Definiamo come IR drop, la consequente caduta di tensione provocata dalla resistenza dei fili di alimentazione. Quando la tensione di alimentazione di un transistor diminuisce, diventa più lento e ciò potrebbe influire sul *timing* del circuito. Quando ciò si verifica su un percorso critico, può anche portare a un guasto funzionale e quindi deve essere evitato. Inoltre variazioni della tensione di alimentazione possono potenzialmente portare alla memorizzazione di valori logici errati.

Un secondo problema che si intreccia con l'IR drop è l'elettromigrazione. Quando grandi correnti fluiscono attraverso i fili sottili di alimentazione, può verificarsi uno spostamento molecolare lungo diverse linee metalliche. Questo provoca un ulteriore restringimento del filo in alcuni determinati punti e quindi con un conseguente aumento della sua resistenza. Nel tempo questo effetto continua a peggiorare l'IR drop.

L'IR drop può avere un impatto ancora maggiore sui circuiti analogici. Oltre ad effetti sul *timing* del circuito, può anche causare direttamente guasti funzionali. Dato che l'assorbimento di corrente di un filo è influenzato dall'attività in altre aree del chip, è spesso visto come una fonte di disturbo per i circuiti analogici e questo deve essere preso in considerazione quando si esegue la progettazione del circuito.

Capitolo 3

Simulazioni e misure sul Read-out tree

Come accennato nel capitolo precedente, per capire meglio le potenziali cause del malfunzionamento del Read-out Tree(ROT) nelle verifiche sperimentali, abbiamo effettuato diverse simulazioni per prendere in considerazione diversi effetti che potrebbero generare il malfunzionamento osservato. In questo capitolo verranno descritti tutti i risultati osservati dalle varie simulazioni. Per prima cosa, si deve capire il processo di progettazione di un circuito integrato digitale.

La prima fase di progettazione del circuito, nota come **modellizzazione**[8], è la fase di descrizione comportamentale del circuito, prescindendo dalla sua implementazione. Per il circuito Timespot1, è stato utilizzato il linguaggio di descrizione hardware VHDL (VHSIC HDL, Very High Speed Integrated Circuits Hardware Description Language). La fase successiva è quella della **sintesi** del circuito. La sintesi architetturale consiste nel costruire la struttura macroscopica di un sistema digitale a partire dalla sua descrizione comportamentale. La sintesi viene effettuata dal sintetizzatore di Synopsys, generando una logica RTL (Register Transfer Level) dalla composizione del codice di descrizione comportamentale in VHDL e codice di *constraint* ".sdc" (Synopsys design constraint) in cui vengono definiti i segnali di clock, valore massimo di jitter, ecc. Dal modello precedente astratto, si crea un nuovo modello più dettagliato che ha la caratteristica di includere informazioni necessarie alla fabbricazione.

La terza fase consiste nel disegno del **Layout** del circuito. In questa fase, la struttura prodotta durante la sintesi viene convertita in un design fisico integrato nell'ASIC. Questa fase è nota anche come *place and route*. Dopodiché si valutano i tempi di propagazione dovuti ai componenti parassiti (resistenza e capacità) per

ciascuna linea di circuito oltre ad altri *constraint* aggiuntivi. I tempi di propagazione delle varie interconnessioni si ricavano da simulazioni in primetime di synopsys, considerando combinazioni estreme di parametri fisici che vengono accettati nella fase di fabbricazione del circuito. Queste combinazioni di parametri fisici sono anche noti come *corner*. I tempi di propagazione estratti per ciascuna linea di interconnessione vengono salvati in file ".sdf" (structured data file). I file generati sono 3: Il file *Min* corrisponde al caso in cui la velocità di trasmissione dei dati è massima. Il file *Max* invece corrisponde al caso in cui la velocità di trasmissione dei dati è minima. Mentre il file *Typ* corrisponde ai valori tipici dei ritardi. Effettuare le simulazioni nei vari *corner* è fondamentale per garantire il funzionamento del più alto numero di chip a prescindere dalla variabilità dei parametri di fabbricazione della fetta di silicio.

Prima della fabbricazione del circuito, si effettuano delle simulazioni per verificare il corretto funzionamento del circuito. La fase di **simulazione** consiste nell'analizzare alcune variabili di riferimento durante un certo intervallo di funzionamento del sistema per uno specifico insieme di stimoli di ingresso. Con la verifica che il circuito funzioni correttamente in tutte le condizioni di funzionamento, si conclude la progettazione del circuito. Dopo aver progettato il circuito, si procede con la sua fabbricazione. Una volta fabbricato, si eseguono diversi test di verifica per accertare che il circuito sia privo di errori progettuali.
Per quanto riguarda i test di verifica sperimentali del circuito integrato di Timespot1, si sono verificati errori inattesi nel trasferimento di dati nel Read-out tree (ROT) del circuito, come descritto nella sezione 2.2. Per questo motivo, come lavoro di tesi, c'era bisogno di analizzare approfonditamente, via ulteriori simulazioni, i possibili effetti contribuenti alle osservazioni sperimentali.

Per ogni effetto descritto nella sezione 2.3, si è scritto un codice di *testbench* nel linguaggio di descrizione hardware VERILOG. Per prima cosa sono state svolte simulazioni post-sintesi, che corrispondono a simulazioni di test effettuati dopo la fase di sintesi del progetto senza considerare effetti dovuti agli elementi parassiti del circuito. Dopodiché sono state svolte simulazioni post-layout, considerando gli effetti dovuti agli elementi parassiti del circuito, grazie ai tempi di propagazione estratti nei file ".sdf" con il tool Primetime di Synopsys.

Nelle sezioni seguenti, verranno descritte in dettaglio: l'implementazione delle simulazioni per ciascuno degli effetti elencati nella sezione 2.3 e le conseguenti osservazioni.

3.1 Simulazioni sul ROT evidenziando l'effetto di jitter casuale e skew

3.1.1 Simulazioni post-sintesi con jitter casuale

La fase di simulazione si inizia con la scrittura di un *testbench*. Il *testbench* è costituito da codice scritto in VERILOG, il quale scopo è di generare input nel circuito sintetizzato per poi verificare i conseguenti output generati. La creazione del file di *testbench* parte con la dichiarazione di un *Module* in cui descrivere gli stimoli della simulazione.

```
module Timespot1_Bottom_all_tb ();

    //codice di testbench

endmodule : Timespot1_Bottom_all_tb
```

All'interno del module vuoto, si dichiarano le variabili per tutti i segnali di input e output del ROT. Questi segnali vengono poi istanziati per connetterli direttamente al *design* del circuito sintetizzato. Una volta istanziati i segnali, si generano i segnali di clock e RESET.

Per generare il clock in cui è presente del jitter di 640 Hz in VERILOG, si usano:

- la keyword `always`:

  crea un loop infinito per eseguire in continuo, durante tutta la durata della simulazione, il codice presente al suo interno.

- L'operatore di time delay `#` :

  Per simulare un time delay in una determinata unità di tempo. La scala temporale viene fissata all'inizio del file con la seguente riga di codice: `` `timescale 1ns / 1ps ``

  Nelle nostro caso abbiamo fissato l'unità di tempo di a 1 ns e la risoluzione a 1 ps.

- la funzione `$dist_normal(seed,media,dv_st)` di VERILOG : *genera un numero casuale secondo una probabilità di distribuzione gaussiana con una determinata deviazione standard, intorno a un valore medio fornito.*

Il clock che presenta del jitter di 640 MHz viene generato con le seguenti linee di codice:

```
always begin
    #(0.78125+($dist_normal(seed,0,sigma))/1000.0)
    CLK640= ~CLK640;
end
```

La variabile `sigma`, presente nel codice sopra riportato, rappresenta il valore del jitter fissato in ps. Il valore numerico prodotto dalla funzione `$dist_normal(seed,0, sigma)` viene diviso per 1000 in modo da restituire un numero in unità temporale di nanosecondi (ns).

Sono state eseguite diverse simulazioni, modificando il valore fissato di jitter, passando da `sigma=50` fino `sigma=300`. In ognuno di questi casi si è simulato l'effetto di questo clock, che presenta del jitter, sulla trasmissione dei dati in uscita del ROT. Come descritto nella sezione 1.2.3, l'output del ROT per ogni *hit* è costituito dall'unione di 8 bit di Address, 23 bit provenienti dal TDC(Time to Digital Converter), e 9 bit di Timestamp. Nel testbench, si sono generati gli 8 bit di address e i 23 bit provenienti dal TDC per ogni *hit*, definendo una funzione `writeTDCTS(address, TDC)` che associa un address e un valore di TDC ai segnali `address[7:0]` e `OUT\_TDC[22:0]`, istanziati precedentemente nel testbench.

Nelle seguenti linee di codice, troverete una parte dei valori degli input(address in decimali e TDC in esadecimali) generati nel testbench:

```
#100
    @(posedge rCLK40)
     writeTDCTS(286, 23'h0AA271);
#100
    @(posedge rCLK40)
     writeTDCTS(287, 23'h0AA272);
```

```
#100
    @(posedge rCLK40)
     writeTDCTS(288, 23'h0AA273);
#100
    @(posedge rCLK40)
     writeTDCTS(289, 23'h0AA275);
#100
    @(posedge rCLK40)
     writeTDCTS(290, 23'h0AA276);
#100
    @(posedge rCLK40)
     writeTDCTS(291, 23'h0AA277);
#100
    @(posedge rCLK40)
     writeTDCTS(292, 23'h0AA278);
#100
    @(posedge rCLK40)
     writeTDCTS(293, 23'h0AA279);
#100
    @(posedge rCLK40)
     writeTDCTS(294, 23'h0AA290);
#100
    @(posedge rCLK40)
     writeTDCTS(295, 23'h0AA291);
            .
            .
            .
            .

#100
    @(posedge rCLK40)
    writeTDCTS(20, 23'h00020);
#500
    @(posedge rCLK40)
     fork
        writeTDCTS(3, 23'h0ABE3A);
        writeTDCTS(4, 23'h0ABE4A);
```

```
        writeTDCTS(5, 23'h0ABE5A);
        writeTDCTS(9, 23'h0ABE9A);
        writeTDCTS(14, 23'h0BE14A);
        writeTDCTS(15, 23'h0FFFFF);
            .
            .
            .
            .
    join
```

Come si può vedere nelle linee di codice sopra riportate, gli input dei 23 bit uscenti dal TDC sono rappresentati in esadecimali con un prefisso di `23'h`. Il prefisso `23'h` indica che l'input è di 23 bit ed è rappresentato in esadecimali.

Dopo la conclusione di scrittura del testbench, si iniziano le simulazioni-post sintesi. Le simulazioni sono state eseguite con l'utilizzo di un software denominato VERDI. Nelle prime simulazioni, si è analizzato l'effetto di diversi valori di jitter (`sigma`) sulla trasmissione dei dati in uscita dal ROT. In particolare si sono confrontati i valori di `rdata[39:0]` (40 bit entranti in una delle Fifo) e `Int_byte_output[7:0]`(Pachetti di 8 bit uscenti dalle Fifo) con i valori di input inseriti nel testbench. Si sono eseguite diverse simulazioni con i seguenti valori di `sigma`: 50 ps, 100 ps, 150 ps, 200 ps, 250 ps, 300 ps. Per ognuno di questi casi, i dati vengono trasmessi correttamente nonostante il jitter crescente. Gli output del ROT rispettavano l'andamento atteso dal testbench. Nella Figura 3.1 si può osservare, da una simulazione con `sigma=100`, la corrispondenza tra gli output del ROT e gli input inseriti nelle linee di codice sopra riportate.

3.1.2 Simulazioni post-layout con jitter casuale e skew

Nelle simulazioni post-layout, si è tenuto conto anche degli effetti temporali dei parassiti e corner del circuito, precedentemente estratti in file ".sdf".

```
$sdf_annotate("/../Bottom_all_v2_final_v1_typ.sdf",dut,,,"MAXIMUM");
//$sdf_annotate("/../Bottom_all_v2_final_v1_min.sdf",dut,,,"MAXIMUM");
//$sdf_annotate("/../Bottom_all_v2_final_v1_max.sdf",dut,,,"MAXIMUM");
```

Queste righe di codice aggiungono archi temporali ad ogni connessione del circuito integrato, compreso l'albero di clock, impattando sullo skew del clock e sulla propagazione dei segnali.

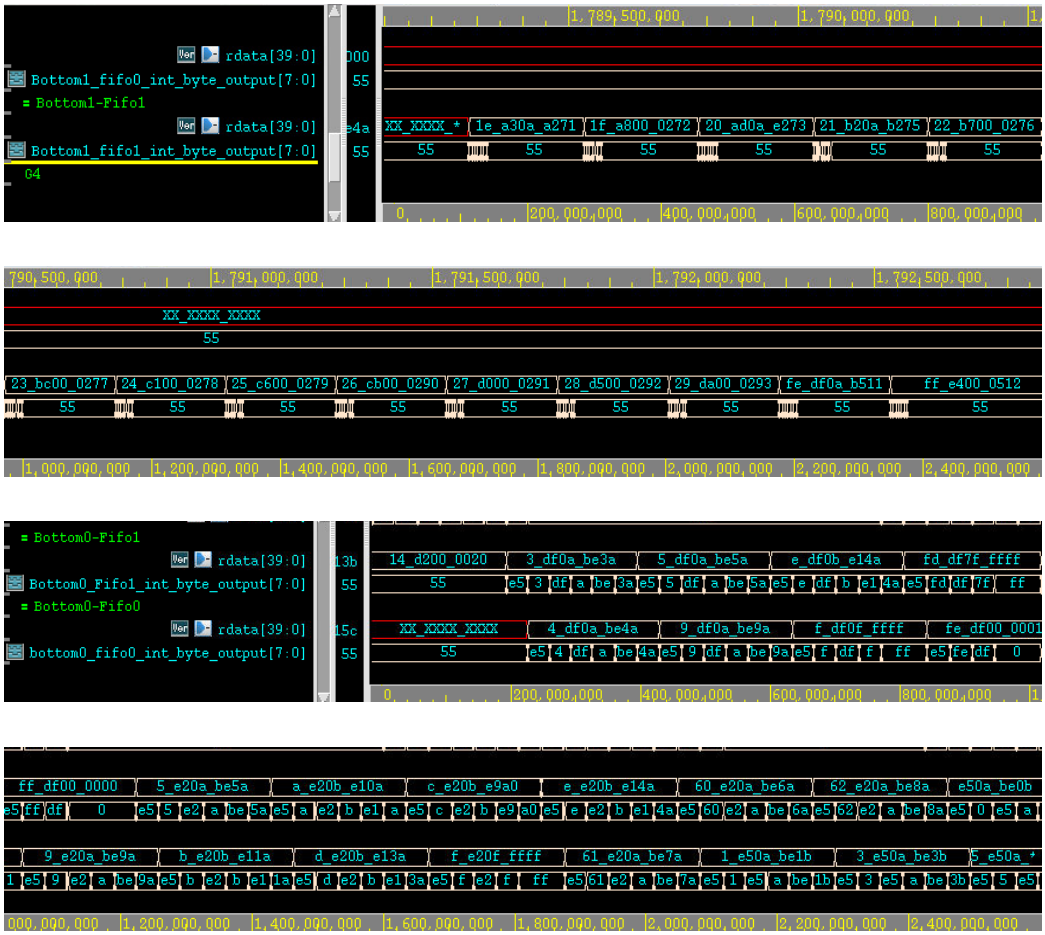


Figura 3.1: Alcuni degli output di dati in uscita del ROT in una simulazione con jitter($sigma$) di 100 ps

Per ogni corner associato, si sono eseguite le stesse simulazioni usando clock con jitter da `sigma=50` a `sigma=250` senza aver trovato differenze rispetto ai risultati delle simulazioni post-sintesi. Nelle simulazioni con `sigma=300` si sono invece osservate differenze specificamente nei corner *typical* e *maximum*. Ci sono stati alcuni dati non trasmessi in maniera corretta. Un risultato normale dato l'elevato jitter rispetto al jitter di 50 ps impostato nel codice di *constraint* durante la fase di progettazione del circuito integrato.

Come descritto nella sezione 2.2, c'è la necessità di analizzare i 4 segnali `OUT_LVDS[3:0]` in uscita dal ROT. Infatti nei test sperimentali si sono osservate alcune perdite di bit oltre a restringimenti temporali dei bit "0". Per verificare la presenza o meno di bit perduti nel segnale di *idle* trasmesso in uscita dagli LVDS, lo si deve confrontare con un segnale atteso prodotto dal testbench. Quindi si è creata una funzione che genera il byte "AB" ("10101011", sequenza scelta inizialmente per l'*idle*) ogni 160 Mhz.

```
task writeAB;
    input [7:0] ab;
    begin
            @(negedge CLK640)
            #0.150
            AB=ab[7];
            @(posedge CLK640)
            #0.150
            AB=ab[6];
            @(negedge CLK640)
            #0.150
            AB=ab[5];
            @(posedge CLK640)
            #0.150
            AB=ab[4];
            @(negedge CLK640)
            #0.150
            AB=ab[3];
            @(posedge CLK640)
            #0.150
            AB=ab[2];
            @(negedge CLK640)
            #0.150
```

```
            AB=ab[1];
            @(posedge CLK640)
            #0.150
            AB=ab[0];
    end
endtask

always begin
    AB=writeAB(8'hAB);
    //AB=writeAB(8'h55);
end
```

`@(posedge CLK640)` indica il momento del prossimo fronte di salita del clock a 640MHz e `@(negedge CLK640)` indica quello del fronte discendente del clock. L'operatore di time delay # è usato per sincronizzare i 2 segnali oggetto di confronto. Con la funzione definita, si possono anche generare altri segnali in base alla "parola" utilizzata per definire l'*idle*. Nel nostro caso, sono state effettuate simulazioni con idle "AB" ("10101011") e "55" ("01010101").

Per evidenziare le differenze nel tempo fra il segnale generato e quello in uscita dagli LVDS, si è applicato l'operatore logico XOR. L'*or esclusivo*, XOR, è un operatore logico che restituisce VERO ("1") se, e solo se, uno solo dei suoi operandi è VERO. Per la tabella di verità dell'operatore XOR, si veda la tabella 3.1.

Tabella 3.1: Tabella di verità dell'operatore XOR

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Con l'operatore logico XOR, si può creare un segnale che restituisce "1" solo nelle aree temporali in cui i 2 segnali operandi presentano differenze. Si è definito un segnale `verifica[3:0]` con le seguenti righe di codice, dove ^ rappresenta l'operatore XOR.

```
assign verifica[3]=AB^OUT_LVDS[3];
assign verifica[2]=AB^OUT_LVDS[2];
assign verifica[1]=AB^OUT_LVDS[1];
assign verifica[0]=AB^OUT_LVDS[0];
```

Per stabilire se le differenze siano veramente dovute ad una perdita di bit e non solo a variazioni dovute al jitter, i valori di `verifica[3:0]` sono stati registrati nel vettore `v[3:0]` ad ogni fronte di salita di `CLK_640`.

```
reg[3:0] v;
    always @(CLK640_TEST)begin
            v[3]=verifica[3];
            v[2]=verifica[2];
            v[1]=verifica[1];
            v[0]=verifica[0];
    end
```

In questo modo, se ci fosse una perdita di bit nell'*idle* di almeno una delle LVDS, il registro definito `v[3:0]` presenterebbe un valore non nullo in quella zona temporale.

Eseguendo le simulazioni di clock con jitter, con valori di `sigma` tra 50 ps e 300 ps, non si osserva alcuna perdita di bit. Nei segnali di *idle* in uscita dagli LVDS, non si osservano variazioni significative nella forma del segnale, ma solo variazioni relative al jitter, come in Figura 3.2. Nelle Figure 3.3 e 3.4 potete vedere i risultati di una delle verifiche del corretto trasferimento di un *idle* "55".

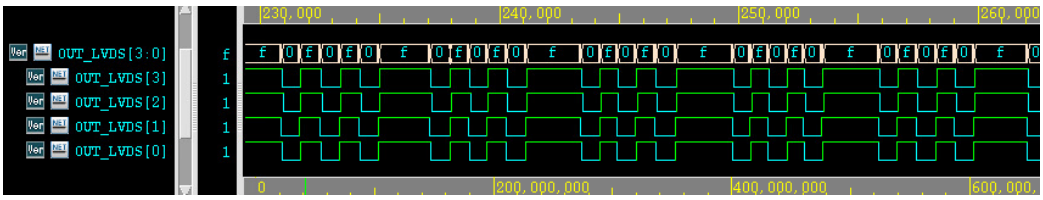


Figura 3.2: Segnali di *idle* in uscita dai 4 LVDS, in una simulazione di jitter casuale(100ps)

Inoltre, si sono eseguite diverse simulazioni, aumentando ulteriormente i ritardi dei corner in vari punti del circuito, modificando i file di ritardo, senza mai trovare

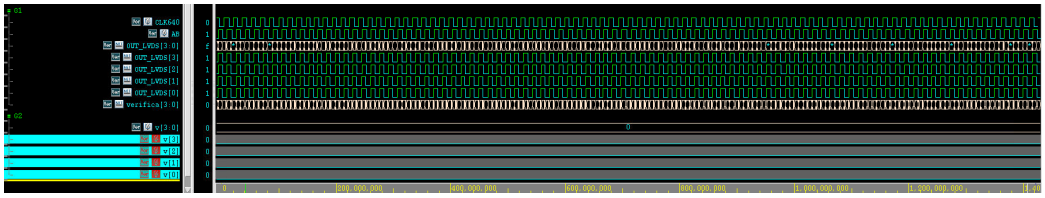


Figura 3.3: Risultati di verifica dell'*idle* "55" in una simulazione di jitter casuale(100ps).

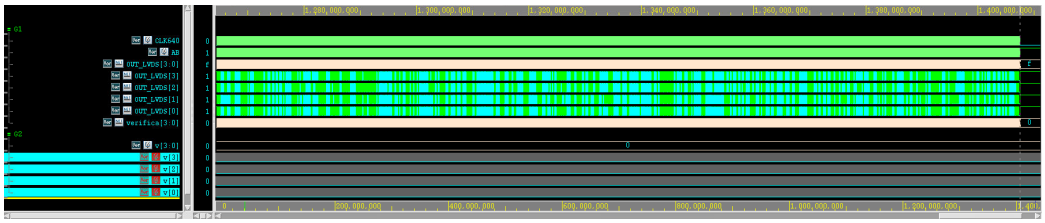


Figura 3.4: Risultati di verifica dell'*idle* "55" in una simulazione di jitter casuale(100ps). Si noti come v[3:0] è uguale a 0 su tutta la linea temporale, confermando nessuna perdita di bit .

un risultato simile alle osservazioni sperimentali mostrate in Figura 2.3. Sono stati analizzati i seguenti punti sensibili del circuito:

- **Interconnessioni connesse al punto di generazione del** `CLK640`: Aumentando il ritardo oltre un certo limite, il segnale `CLK640` non è più riconoscibile come un segnale periodico di clock. Non si è trovata una corrispondenza tra le osservazioni di simulazione e di test sperimentale.

- **Interconnessione nel punto di entrata del** `CLK640` **nel Bottom 0**: Aumentando il ritardo in questo punto del circuito, si è verificato un errore nel trasferimento dei dati. Situazione non coincidente con le osservazioni sperimentali.

- **Interconnessione nel punto di entrata del** `CLK640` **nel Bottom 1**: Non si è osservata una differenza rispetto alle simulazioni precedenti di aumento del ritardo all'entrata del Bottom 0.

- **Interconnessioni precedenti alle 2 componenti DDR(Double Data Rate) del Bottom 0**: Aumentando il ritardo si è osservato un glitch nella forma dell'*idle* "AB" in uno dei segnali `OUT_LVDS`, come in figura 3.5. Non si sono osservati i restringimenti degli "0" presenti nei test sperimentali.

- **Interconnessioni precedenti alle 2 componenti DDR(Double Data Rate) del Bottom 1**: Si sono osservati gli stessi risultati delle simulazioni del punto precedente.

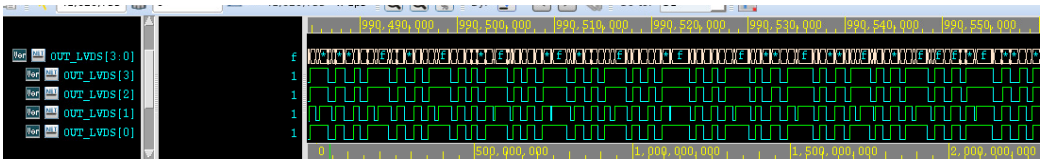


Figura 3.5: Segnali LVDS in una simulazione con aggiunta di ritardo di in una interconnessione precedente alla componente DDR (*ddr_adapter/d1_q*) connessa alla FIFO 1 del Bottom 1.

3.2 Simulazioni sul ROT evidenziando il duty cycle del clock

In questa sezione, si trovano i risultati delle simulazioni sul ROT in cui si sono variati i duty cycle del `CLK640`. Il termine duty cycle viene usato per indicare il rapporto tra la durata del segnale "1" e il periodo totale del clock, ed è espresso in percentuali.

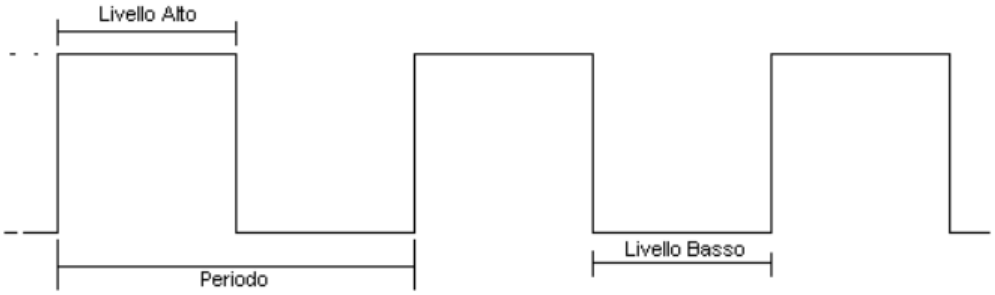


Figura 3.6: Esempio di onda quadra

Facendo riferimento all'onda quadra nella figura 3.6, il duty cycle viene ricavato con la seguente formula[9]:

$$\text{Duty cycle}(\%) = \frac{LivelloAlto}{Periodo} \cdot 100$$

Un clock uniforme ha un duty cycle del 50%. Nelle nostre simulazioni, si è valutato l'effetto, sulla trasmissione dell'*idle* "AB" in uscita dagli LVDS, di clock

con diverse percentuali di duty cycle. In particolare, si sono generati vari clock con duty cycle compreso tra il 56,4% al 78,8%. Nelle seguenti righe di codice trovate un esempio di definizione di un clock con duty cycle del 72,4% in VERILOG.

```
always begin
    #(0.78125 - 0.35 + ($dist_normal(seed, 0, sigma))/1000.0)
    CLK640 = ~CLK640;
    #(0.78125 + 0.35 + ($dist_normal(seed, 0, sigma))/1000.0)
    CLK640 = ~CLK640;
end
```

Nelle righe di codice sopra riportate, si sono accorciati gli "0" e allungati gli "1" entrambi di 350 ps, mantenendo invariato il periodo del `CLk640` di 1.5625 nm.

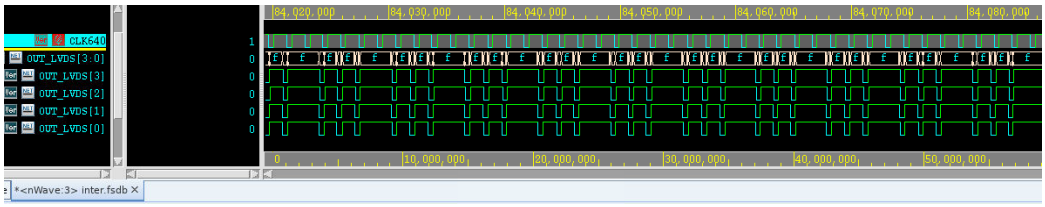


Figura 3.7: Segnali di *idle* in uscita dagli LVDS, in una simulazione con duty cycle del 72,4% e jitter casuale di 50 ps

Come si può vedere nella figura 3.7, all'aumentare del duty cycle si osserva un restringimento degli "0" del segnale *idle* "AB" in uscita dagli LVDS. Inoltre, aumentando il duty cycle oltre il 79% (± 450ps nella definizione di clock in VERILOG) si osserva anche una perdita bit, come in figura 3.8. Si può notare come i segnali dell'*idle* in uscita dagli LVDS siano molto simili alle osservazioni sperimentali in figura 2.3.

3.3 Analisi e misure relative all'effetto IR Drop

Come accennato nella sezione 1.2.3, i byte di dati in uscita dalle FIFO del ROT vengono serializzati e convertiti in Double Data Rate(DDR) per poi essere trasmessi in output dai driver LVDS ad una velocità di 1.28 Gbit/s. Un segnale viene trasmesso in modo DDR quando i dati vengono trasmessi su entrambi i fronti del clock, raddoppiando così la velocità di trasmissione dei byte di dati. In ogni blocco

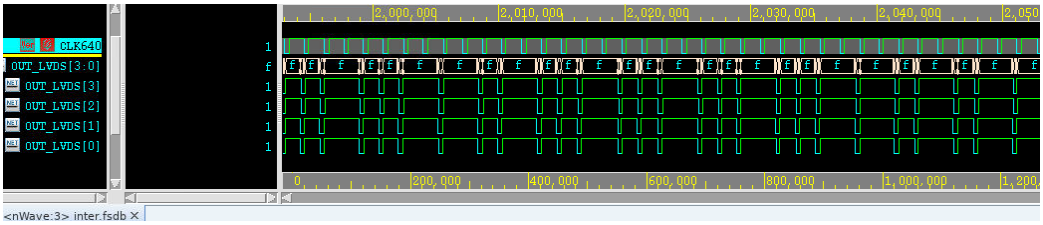


Figura 3.8: Segnali di *idle* in uscita dagli LVDS, in una simulazione con duty cycle del 78,8% e jitter casuale di 50 ps

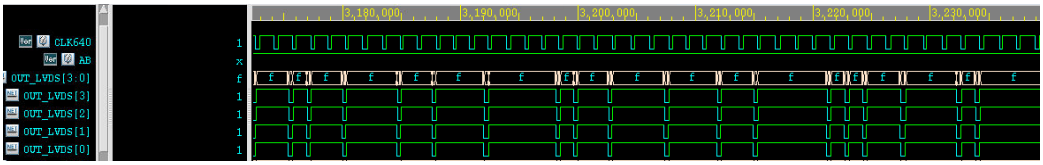


Figura 3.9: Segnali di *idle* in uscita dagli LVDS, in una simulazione con duty cycle del 78,8% e jitter casuale di 100 ps

ROT, sono presenti 2 circuiti che convertono da SDR(Single Data Rate) a DDR, ciascuno associato ad una delle 2 FIFO.

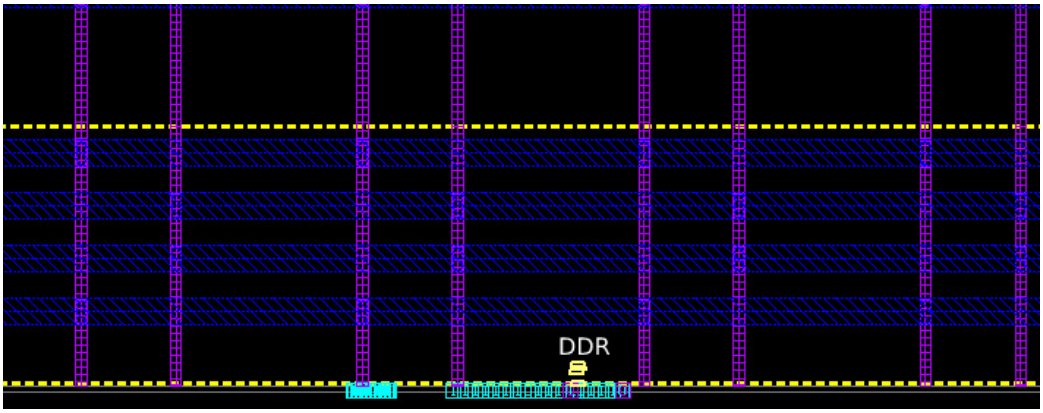


Figura 3.10: Posizione delle componenti DDR in uno dei 2 bottom, evidenziando con i colori blu e viola i diversi strati metallici di alimentazione nel software IC LAYOUT EDITOR

In questa sezione, analizziamo le possibili differenze di alimentazione delle componenti DDR di ognuno dei 2 bottom. In particolare, utilizzando il software IC LAYOUT EDITOR, si sono misurate le distanze delle componenti DDR, di entrambi i bottom, dai rispettivi strati metallici di alimentazione più vicini, come in Figura 3.10. Misurando le distanze, si possono ricavare le resistenze delle linee

di connessione oltre alle conseguenti cadute di tensione(IR Drop).

Nella seguente tabella, trovate le misure delle distanze tra gli strati di metallo di alimentazione e le componenti DDR, per ciascuno dei Bottom.

Tabella 3.2: Distanze misurate delle DDR dai strati metallici di alimentazione

	Bottom 0	**Bottom 1**
Distanza da sinistra(in µm)	24	24
Distanza da destra(in µm)	11	11

Dalle misure delle distanze, in tabella 3.2, non si trovano differenze di alimentazione, legate all'IR drop, nelle componenti DDR dei 2 bottom.

Capitolo 4

Conclusioni

Lo scopo principale di questa tesi è stato di individuare la causa della corruzione dei dati osservata, durante i test sperimentali, nelle uscite di 2 dei ROT del circuito integrato Timespot1. A questo scopo sono stati analizzati, con simulazioni digitali, vari effetti possibilmente contribuenti alle osservazioni sperimentali. Per questa tesi si sono considerati gli effetti di jitter e skew sul clock oltre all'effetto di IR drop in determinati nodi del circuito. Le simulazioni sono state effettuate utilizzando il processo di progettazione digitale sviluppato da Synopsys, in particolare il software VERDI e vari testbench scritti in Verilog, un linguaggio di descrizione hardware.

Analizzando le uscite degli LVDS dei ROT, solamente nelle simulazioni digitali incentrate sull'effetto della modifica del duty cycle, si è trovata una correlazione con le osservazioni sperimentali. In particolare quando il duty cycle aumenta oltre il 79%, considerando un jitter casuale di 50 ps, si osserva una perdita di bit oltre a un restringimento dei bit "0" dei segnali in uscita dagli LVDS dei ROT.

Dalle varie simulazioni, non si sono trovate differenze tra le uscite dei 4 ROT. In particolare, con effetti di IR drop non completamente analizzati e dall'impossibilità di simulare alcuni possibili difetti di fabbricazione, non si è riuscito a giungere a una conclusione sulla causa delle differenze di output osservate sperimentalmente tra i "Bottom 0" e i "Bottom 1".

Per i futuri sviluppi si analizzerà in modo più approfondito l'IR drop di tutte le componenti utilizzando un tool informatico attualmente non disponibile. Inoltre, si procederà progettando in modo differente i DDR dei ROT.

Bibliografia

[1] "The large hadron collider," CERN. Available: https://www.home.cern/science/accelerators/large-hadron-collider

[2] A. Schaeffer, "New schedule for CERN's accelerators and experiments," Nov 2020. Available: https://home.cern/news/news/accelerators/new-schedule-cerns-accelerators-and-experiments

[3] "High-luminosity LHC," CERN. Available: https://home.cern/science/accelerators/high-luminosity-lhc

[4] S. Cadeddu, L. Frontini, A. Lai, V. Liberali, L. Piccolo, A. Rivetti, and A. Stabile, "Operation and performance of timespot1: A high time-resolution 28 nm CMOS pixel read-out ASIC," in *2021 IEEE Nuclear Science Symposium and Medical Imaging Conference (NSS/MIC)*, 2021, pp. 1–3.

[5] M. Barbaro *et al.*, "A pixel read-out front-end in 28 nm CMOS with time and space resolution," in *2019 IEEE Nuclear Science Symposium and Medical Imaging Conference (NSS/MIC)*, 2019, pp. 1–4.

[6] S. Procopio, "Preparazione del test del circuito integrato TIMESPOT1," Milano, 2020, tesi di Laurea Magistrale, Università degli studi di Milano.

[7] D. Perelez, "How to track down setup and hold violations with a mixed signal oscilloscope," Oct 2019. Available: https://www.designnews.com/electronics-test/how-track-down-setup-and-hold-violations-mixed-signal-oscilloscope

[8] M. Salmeri, "Le fasi della realizzazione di sistemi elettronici," 2000. Available: https://didattica-2000.archived.uniroma2.it/pa/deposito/pa_cap_2.pdf

[9] M. Lai, "Capire il duty cycle," Jan 2017. Available: https://logicaprogrammabile.it/duty-cycle-guida-definitiva/